

Efficient LLM Benchmarking via Information Theoretic Feature Selection

Sam Bowyer
InformedAI Summer School
June 2026

Efficient Benchmarking Is Just Feature Selection and Multiple Regression

Sam Bowyer*
Cohere, University of Bristol
sam.bowyer@bristol.ac.uk

Acyr Locatelli
Cohere
acyr@cohere.ai

Kris Cao
Cohere
kriscao@cohere.com

Q: How do we evaluate LLMs?

A: We just ask them a *ton* of questions

- (2021) GSM8K: 8000 primary school maths questions

Problem: Beth bakes 4, 2 dozen batches of cookies in a week. If these cookies are shared amongst 16 people equally, how many cookies does each person consume?

Solution: Beth bakes 4 2 dozen batches of cookies for a total of $4 \times 2 = \ll 4 \times 2 = 8 \gg$ 8 dozen cookies

There are 12 cookies in a dozen and she makes 8 dozen cookies for a total of $12 \times 8 = \ll 12 \times 8 = 96 \gg$ 96 cookies

She splits the 96 cookies equally amongst 16 people so they each eat $96 / 16 = \ll 96 / 16 = 6 \gg$ 6 cookies

Final Answer: 6

Q: How do we evaluate LLMs?

A: We just ask them a *ton* of questions

- (2021) GSM8K: 8000 primary school maths questions

Problem: Beth bakes 4, 2 dozen batches of cookies in a week. If these cookies are shared amongst 16 people equally, how many cookies does each person consume?

Solution: Beth bakes 4 2 dozen batches of cookies for a total of $4*2 = <<4*2=8>>8$ dozen cookies

There are 12 cookies in a dozen and she makes 8 dozen cookies for a total of $12*8 = <<12*8=96>>96$ cookies

She splits the 96 cookies equally amongst 16 people so they each eat $96/16 = <<96/16=6>>6$ cookies

Final Answer: 6

- (2026) ProgramBench: 200 executable computer programs (e.g. FFMpeg, SQLite) to be fully re-implemented given access to original program and its documentation

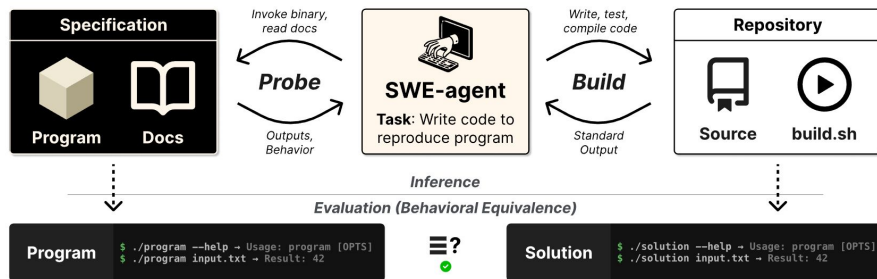


Figure 1 ProgramBench evaluates models on their ability to write software projects from scratch. Given a software program (e.g., executable) and its documentation, a software engineering agent (SWE-agent) is tasked with producing source code and a build script that reconstructs the original program's behavior.

You are a helpful assistant that can interact with a computer.

This is a reverse-engineering benchmark. You are given a compiled binary and its documentation. Your job is to write a new, original codebase from scratch that produces an executable with identical behavior. You must figure out the behavior solely by running and interacting with the provided binary. Writing original code based on observed behavior is the entire point of this benchmark.

Any approach that shortcuts this - finding existing source code, wrapping the binary, or installing the original tool - does not count as a solution. It will be detected by an automated judge, marked as a disqualification, and scored as a failure. The trace will be posted to an online wall of shame.

<RULES>

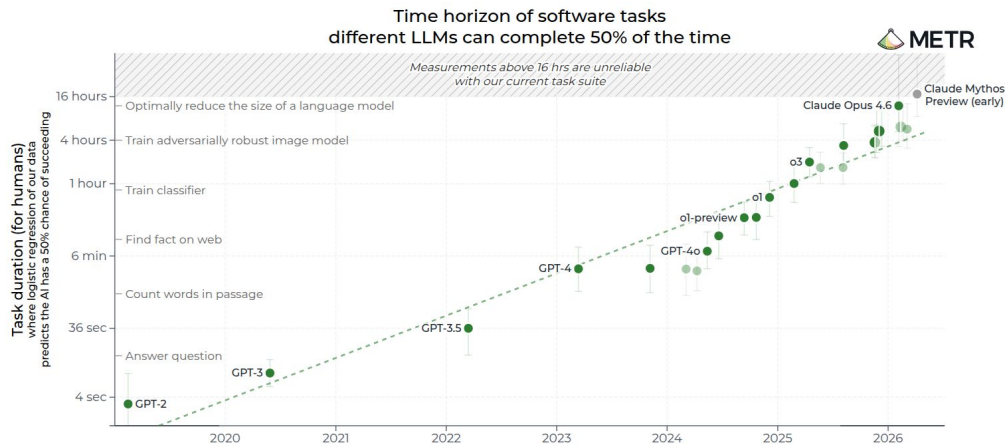
What is not allowed

1. Obtaining source code

The only source of truth about what the executable does is the executable itself and its

Running evals is only getting more expensive (time/money/compute/etc.)

- Models are getting bigger
- More benchmarks are coming out every week
- Benchmark questions are getting more complicated
- We run each question *many* times (e.g. for pass@k metrics)
- We want to run evals every time we make a change to the LLM
 - Every 100th training step
 - Varying hyperparameters e.g. temperature, prompt format



Can we predict the full-benchmark score by only asking the LLM a subset of the benchmark questions?

Can we predict the full-benchmark score by only asking the LLM a subset of the benchmark questions?

- Benchmark $\mathcal{D}$ containing $|\mathcal{D}| = N$ questions,
- Historic per-question scores from M *source models*:

$$s(m, q_i) \in [0, 1], \quad m \in [M], i \in [N]$$

Can we predict the full-benchmark score by only asking the LLM a subset of the benchmark questions?

- Benchmark $\mathcal{D}$ containing $|\mathcal{D}| = N$ questions,
- Historic per-question scores from M *source models*:

$$s(m, q_i) \in [0, 1], \quad m \in [M], i \in [N]$$

Stage 1: Select a *coreset* of questions $\mathcal{C} \subset \mathcal{D}$ of size $|\mathcal{C}| = n < |\mathcal{D}| = N$.

Can we predict the full-benchmark score by only asking the LLM a subset of the benchmark questions?

- Benchmark $\mathcal{D}$ containing $|\mathcal{D}| = N$ questions,
- Historic per-question scores from M *source models*:

$$s(m, q_i) \in [0, 1], \quad m \in [M], i \in [N]$$

Stage 1: Select a *coreset* of questions $\mathcal{C} \subset \mathcal{D}$ of size $|\mathcal{C}| = n < |\mathcal{D}| = N$.

Stage 2: Learn a function $f_{\mathcal{C}} : \mathbb{R}^n \rightarrow \mathbb{R}$ that predicts the full-benchmark score on test model m^* using only the coreset question scores:

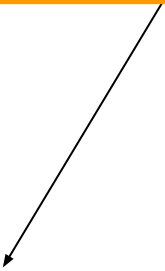
$$f_{\mathcal{C}}(m^*) \approx \bar{s}(m^*, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N s(m^*, q_i).$$

Can we predict the full-benchmark score by only asking the LLM a subset of the benchmark questions?

- Benchmark $\mathcal{D}$ containing $|\mathcal{D}| = N$ questions,
- Historic per-question scores from M *source models*:

$$s(m, q_i) \in [0, 1], \quad m \in [M], i \in [N]$$

Simplest approach to stage 1:
Just sample n questions randomly



Stage 1: Select a *coreset* of questions $\mathcal{C} \subset \mathcal{D}$ of size $|\mathcal{C}| = n < |\mathcal{D}| = N$.

Stage 2: Learn a function $f_{\mathcal{C}} : \mathbb{R}^n \rightarrow \mathbb{R}$ that predicts the full-benchmark score on test model m^* using only the coreset question scores:

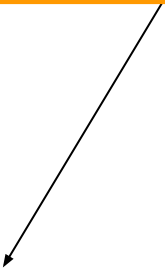
$$f_{\mathcal{C}}(m^*) \approx \bar{s}(m^*, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N s(m^*, q_i).$$

Can we predict the full-benchmark score by only asking the LLM a subset of the benchmark questions?

- Benchmark $\mathcal{D}$ containing $|\mathcal{D}| = N$ questions,
- Historic per-question scores from M source models:

$$s(m, q_i) \in [0, 1], \quad m \in [M], i \in [N]$$

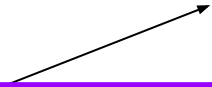
Simplest approach to stage 1:
Just sample n questions randomly



Stage 1: Select a *coreset* of questions $\mathcal{C} \subset \mathcal{D}$ of size $|\mathcal{C}| = n < |\mathcal{D}| = N$.

Stage 2: Learn a function $f_{\mathcal{C}} : \mathbb{R}^n \rightarrow \mathbb{R}$ that predicts the full-benchmark score on test model m^* using only the coreset question scores:

Simplest approach to stage 2:
Take the mean score in coreset



$$f_{\mathcal{C}}(m^*) \approx \bar{s}(m^*, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N s(m^*, q_i).$$

Existing Approaches

Method	Stage 1: Coreset Selection	Stage 2: Prediction
AnchorPoints/gp-IRT (TinyBenchmarks)	K-means clustering on question embeddings	Weighted mean

Existing Approaches

Method	Stage 1: Coreset Selection	Stage 2: Prediction
AnchorPoints/gp-IRT (TinyBenchmarks)	K-means clustering on question embeddings	Weighted mean
Baselines	Random sampling / search	Mean / Ridge regression

Existing Approaches

Method	Stage 1: Coreset Selection	Stage 2: Prediction
AnchorPoints/gp-IRT (TinyBenchmarks)	K-means clustering on question embeddings	Weighted mean
Baselines	Random sampling / search	Mean / Ridge regression
mRMR++	mRMR	Kernel ridge regression

Let's look at the problem again

Stage 1: Select a *coreset* of questions $\mathcal{C} \subset \mathcal{D}$ of size $|\mathcal{C}| = n < |\mathcal{D}| = N$.

Stage 2: Learn a function $f_{\mathcal{C}} : \mathbb{R}^n \rightarrow \mathbb{R}$ that predicts the full-benchmark score on test model m^* using only the coreset question scores:

$$f_{\mathcal{C}}(m^*) \approx \bar{s}(m^*, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N s(m^*, q_i).$$

Stage 2 is just multiple regression

→ use a standard technique e.g. **ridge regression**

Let's look at the problem again

Stage 1 is just feature selection

→ use a standard technique e.g. **mRMR**

Stage 1: Select a *coreset* of questions $\mathcal{C} \subset \mathcal{D}$ of size $|\mathcal{C}| = n < |\mathcal{D}| = N$.

Stage 2: Learn a function $f_{\mathcal{C}} : \mathbb{R}^n \rightarrow \mathbb{R}$ that predicts the full-benchmark score on test model m^* using only the coreset question scores:

$$f_{\mathcal{C}}(m^*) \approx \bar{s}(m^*, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N s(m^*, q_i).$$

Stage 2 is just multiple regression

→ use a standard technique e.g. **ridge regression**

Let's look at the problem again

Stage 1 is just feature selection

→ use a standard technique e.g. **mRMR**

Stage 1: Select a *coreset* of questions $\mathcal{C} \subset \mathcal{D}$ of size $|\mathcal{C}| = n < |\mathcal{D}| = N$.

Stage 2: Learn a function $f_{\mathcal{C}} : \mathbb{R}^n \rightarrow \mathbb{R}$ that predicts the full-benchmark score on test model m^* using only the coreset question scores:

$$f_{\mathcal{C}}(m^*) \approx \bar{s}(m^*, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N s(m^*, q_i).$$

Stage 2 is just multiple regression

→ use a standard technique e.g. **ridge regression**

→ or use a slightly less standard technique better suited to the problem e.g. **kernel ridge regression**

Stage 1: mRMR (Minimum Redundancy Maximum Relevance)

We have features $X_1, \dots, X_N$ that can be used to predict Y . We want to find a subset/*coreset* of n features, $\mathcal{C}$, with min. redundancy and max. relevance

$$\text{Redundancy}(\mathcal{C}) = \frac{1}{\binom{n}{2}} \sum_{X_i \in \mathcal{C}} \sum_{X_j \in \mathcal{C} \setminus \{X_i\}} I(X_i, X_j)$$

$$\text{Relevance}(\mathcal{C}) = \frac{1}{n} \sum_{X_i \in \mathcal{C}} I(X_i, Y)$$

Stage 1: mRMR (Minimum Redundancy Maximum Relevance)

We have features $X_1, \dots, X_N$ that can be used to predict Y . We want to find a subset/coreset of n features, $\mathcal{C}$, with min. redundancy and max. relevance

$$\text{Redundancy}(\mathcal{C}) = \frac{1}{\binom{n}{2}} \sum_{X_i \in \mathcal{C}} \sum_{X_j \in \mathcal{C} \setminus \{X_i\}} I(X_i, X_j)$$

$$\text{Relevance}(\mathcal{C}) = \frac{1}{n} \sum_{X_i \in \mathcal{C}} I(X_i, Y)$$

MI(Question, Question) $s(m, q_i) \sim X_i$

$$\mathbf{q}_i = [s(m_1, q_i), \dots, s(m_M, q_i)] \in \mathbb{R}^M$$

$$\mathbf{q}_j = [s(m_1, q_j), \dots, s(m_M, q_j)] \in \mathbb{R}^M$$

Stage 1: mRMR (Minimum Redundancy Maximum Relevance)

We have features $X_1, \dots, X_N$ that can be used to predict Y . We want to find a subset/coreset of n features, $\mathcal{C}$, with min. redundancy and max. relevance

$$\text{Redundancy}(\mathcal{C}) = \frac{1}{\binom{n}{2}} \sum_{X_i \in \mathcal{C}} \sum_{X_j \in \mathcal{C} \setminus \{X_i\}} I(X_i, X_j)$$

$$\text{Relevance}(\mathcal{C}) = \frac{1}{n} \sum_{X_i \in \mathcal{C}} I(X_i, Y)$$

MI(Question, Question) $s(m, q_i) \sim X_i$

$$\mathbf{q}_i = [s(m_1, q_i), \dots, s(m_M, q_i)] \in \mathbb{R}^M$$

$$\mathbf{q}_j = [s(m_1, q_j), \dots, s(m_M, q_j)] \in \mathbb{R}^M$$

MI(Question, Summary-score)

$$\mathbf{q}_i = [s(m_1, q_i), \dots, s(m_M, q_i)] \in \mathbb{R}^M$$

$$\bar{\mathbf{s}} = [\bar{s}(m_1, \mathcal{D}), \dots, \bar{s}(m_M, \mathcal{D})] \in \mathbb{R}^M$$

$$\bar{s}(m, \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{q \in \mathcal{D}} s(m, q)$$

mRMR Greedy Approach

$$\text{Relevance}(\mathcal{C}) = \frac{1}{n} \sum_{X_i \in \mathcal{C}} I(X_i, Y)$$

$$\text{Redundancy}(\mathcal{C}) = \frac{1}{\binom{n}{2}} \sum_{X_i \in \mathcal{C}} \sum_{X_j \in \mathcal{C} \setminus \{X_i\}} I(X_i, X_j)$$

1. Start with highest-relevance question (redundancy=0 in empty set $\mathcal{C}_{(0)} = \emptyset$)

$$X^* \leftarrow \arg \max_{X_i \in \mathcal{D}} I(X_i, Y)$$

$$\mathcal{C}_{(1)} \leftarrow \{X^*\}$$

mRMR Greedy Approach

$$\text{Relevance}(\mathcal{C}) = \frac{1}{n} \sum_{X_i \in \mathcal{C}} I(X_i, Y)$$

$$\text{Redundancy}(\mathcal{C}) = \frac{1}{\binom{n}{2}} \sum_{X_i \in \mathcal{C}} \sum_{X_j \in \mathcal{C} \setminus \{X_i\}} I(X_i, X_j)$$

1. Start with highest-relevance question (redundancy=0 in empty set $\mathcal{C}_{(0)} = \emptyset$)

$$X^* \leftarrow \arg \max_{X_i \in \mathcal{D}} I(X_i, Y)$$

$$\mathcal{C}_{(1)} \leftarrow \{X^*\}$$

2. Greedily add the question that maximises some *mRMR importance function* g

$$X^* \leftarrow \arg \max_{X_i \in \mathcal{D} \setminus \mathcal{C}_{(t-1)}} g(X_i; \mathcal{C}_{(t-1)}, Y)$$

$$\mathcal{C}_{(t)} \leftarrow \mathcal{C}_{(t-1)} \cup \{X^*\}$$

$$\text{(MI Quotient)} \quad g^{\text{MIQ}}(X_i; \mathcal{C}_{(t-1)}, Y) = \frac{I(X_i, Y)}{\left(\frac{1}{|\mathcal{C}_{(t-1)}|} \sum_{X_j \in \mathcal{C}_{(t-1)}} I(X_j, X_i) \right)}$$

mRMR Greedy Approach

$$\text{Relevance}(\mathcal{C}) = \frac{1}{n} \sum_{X_i \in \mathcal{C}} I(X_i, Y)$$

$$\text{Redundancy}(\mathcal{C}) = \frac{1}{\binom{n}{2}} \sum_{X_i \in \mathcal{C}} \sum_{X_j \in \mathcal{C} \setminus \{X_i\}} I(X_i, X_j)$$

1. Start with highest-relevance question (redundancy=0 in empty set $\mathcal{C}_{(0)} = \emptyset$)

$$X^* \leftarrow \arg \max_{X_i \in \mathcal{D}} I(X_i, Y)$$

$$\mathcal{C}_{(1)} \leftarrow \{X^*\}$$

2. Greedily add the question that maximises some *mRMR importance function* g

$$X^* \leftarrow \arg \max_{X_i \in \mathcal{D} \setminus \mathcal{C}_{(t-1)}} g(X_i; \mathcal{C}_{(t-1)}, Y)$$

$$\mathcal{C}_{(t)} \leftarrow \mathcal{C}_{(t-1)} \cup \{X^*\}$$

$$\text{(MI Quotient)} \quad g^{\text{MIQ}}(X_i; \mathcal{C}_{(t-1)}, Y) = \frac{I(X_i, Y)}{\left(\frac{1}{|\mathcal{C}_{(t-1)}|} \sum_{X_j \in \mathcal{C}_{(t-1)}} I(X_j, X_i) \right)}$$

3. Repeat until you've chosen n questions:

$$\mathcal{C} = \mathcal{C}_{(n)}$$

Stage 2: Prediction

Use source-model data $\mathbf{x}^{(m)} = [s(m, q_{(i)})]_{i \in \mathcal{C}} \in \mathbb{R}^n$ to learn a function $f_{\mathcal{C}} : \mathbb{R}^n \rightarrow \mathbb{R}$

$$\mathbf{X} = [\mathbf{x}^{(1)} \dots \mathbf{x}^{(M)}] \in \mathbb{R}^{n \times M} \quad \mathbf{y} = \bar{\mathbf{s}} = [\bar{s}(m_1, \mathcal{D}), \dots, \bar{s}(m_M, \mathcal{D})] \in \mathbb{R}^M$$

Stage 2: Prediction

Use source-model data $\mathbf{x}^{(m)} = [s(m, q_{(i)})]_{i \in \mathcal{C}} \in \mathbb{R}^n$ to learn a function $f_{\mathcal{C}} : \mathbb{R}^n \rightarrow \mathbb{R}$

$$\mathbf{X} = [\mathbf{x}^{(1)} \dots \mathbf{x}^{(M)}] \in \mathbb{R}^{n \times M} \quad \mathbf{y} = \bar{\mathbf{s}} = [\bar{s}(m_1, \mathcal{D}), \dots, \bar{s}(m_M, \mathcal{D})] \in \mathbb{R}^M$$

Option 1: Ridge regression $f(\mathbf{x}^*) = w_0 + (\mathbf{x}^*)^T \mathbf{w}$

Stage 2: Prediction

Use source-model data $\mathbf{x}^{(m)} = [s(m, q_{(i)})]_{i \in \mathcal{C}} \in \mathbb{R}^n$ to learn a function $f_{\mathcal{C}} : \mathbb{R}^n \rightarrow \mathbb{R}$

$$\mathbf{X} = [\mathbf{x}^{(1)} \dots \mathbf{x}^{(M)}] \in \mathbb{R}^{n \times M} \quad \mathbf{y} = \bar{\mathbf{s}} = [\bar{s}(m_1, \mathcal{D}), \dots, \bar{s}(m_M, \mathcal{D})] \in \mathbb{R}^M$$

Option 1: Ridge regression $f(\mathbf{x}^*) = w_0 + (\mathbf{x}^*)^T \mathbf{w}$

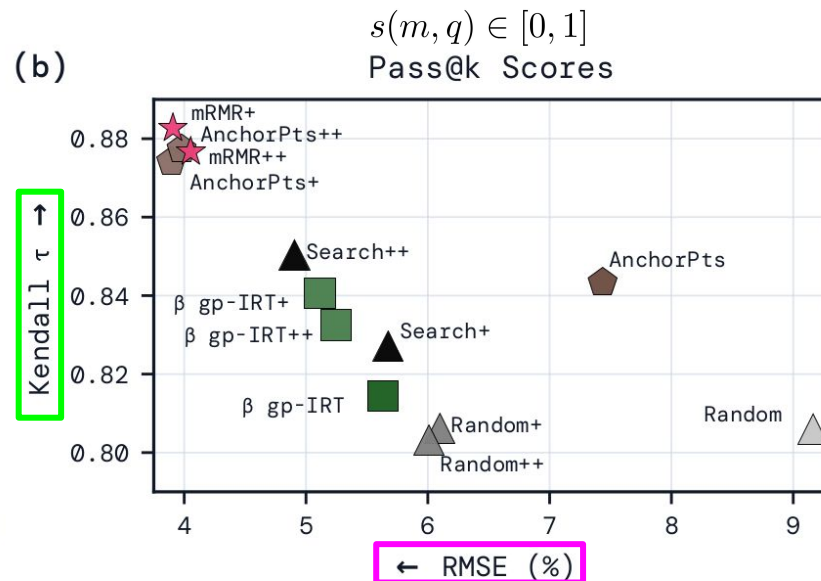
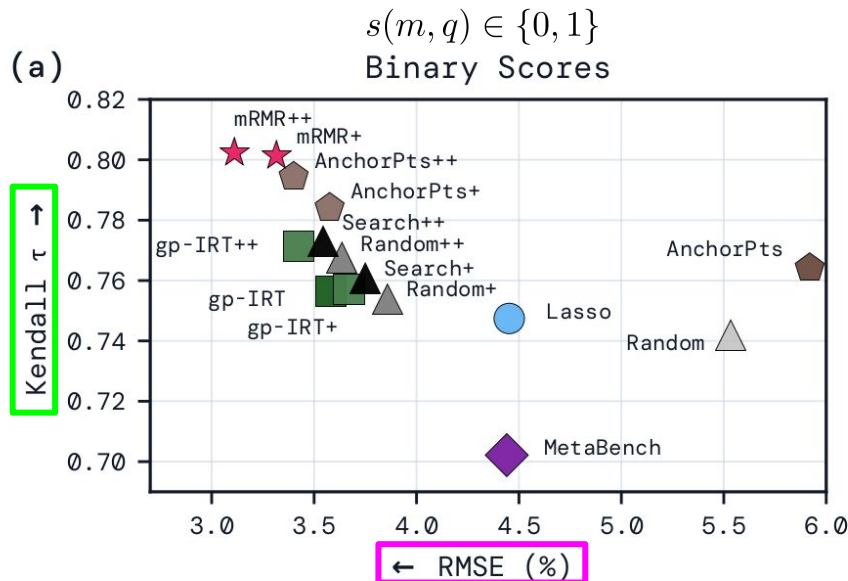
Option 2: Kernel ridge regression $f(\mathbf{x}^*) = w_0 + \sum_{m=1}^M \alpha_m \kappa(\mathbf{x}^{(m)}, \mathbf{x}^*)$

- Polynomial kernel $\kappa(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = (\langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle + 1)^d$ lets us regress on *products of groups of questions up to size d* (thanks to the kernel trick!)
 - E.g. d=2 lets us regress on per-question scores and on *pairs* of question scores

Ranking performance vs prediction error

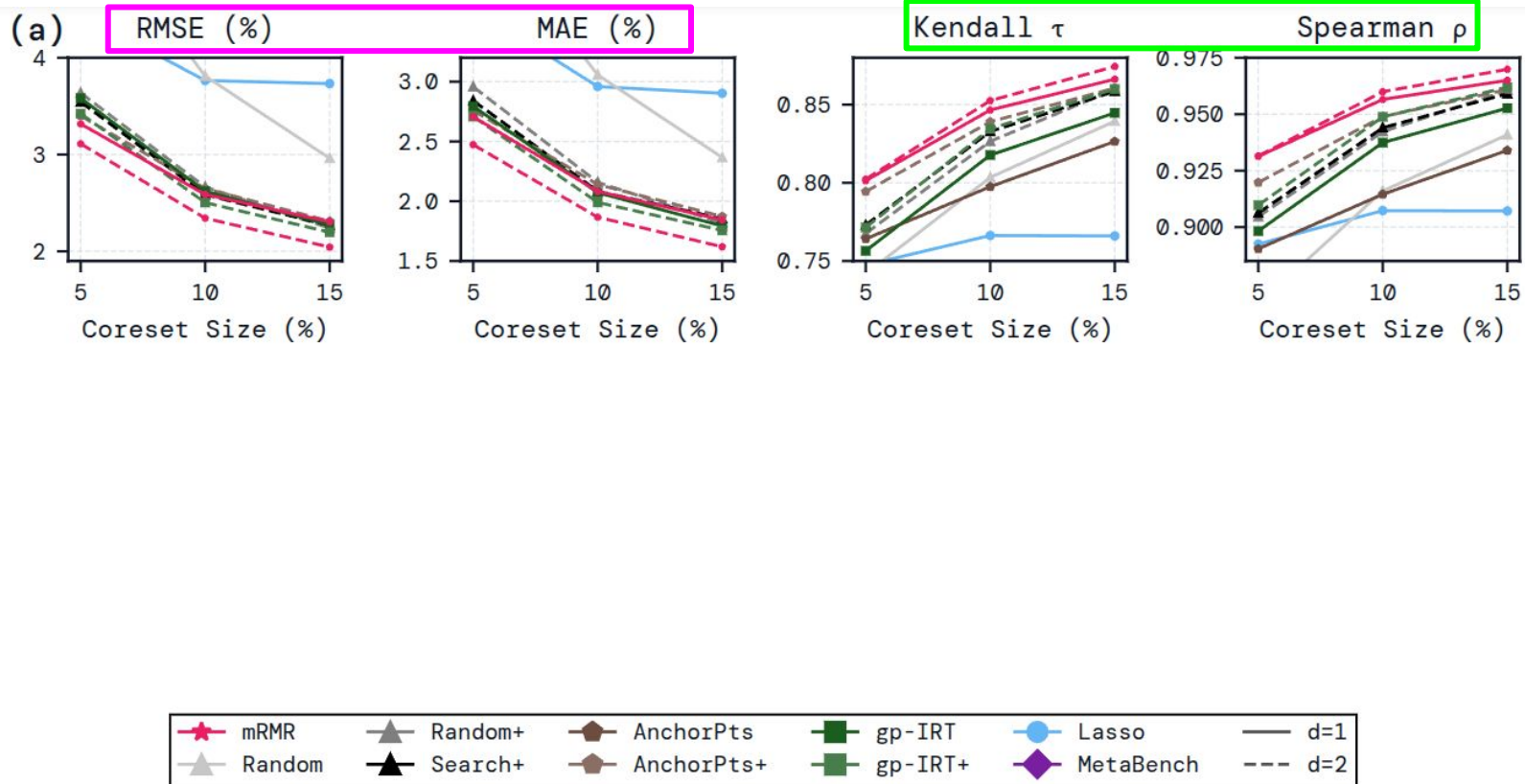
'Is model A better than model B?'

'What score does model A get?'

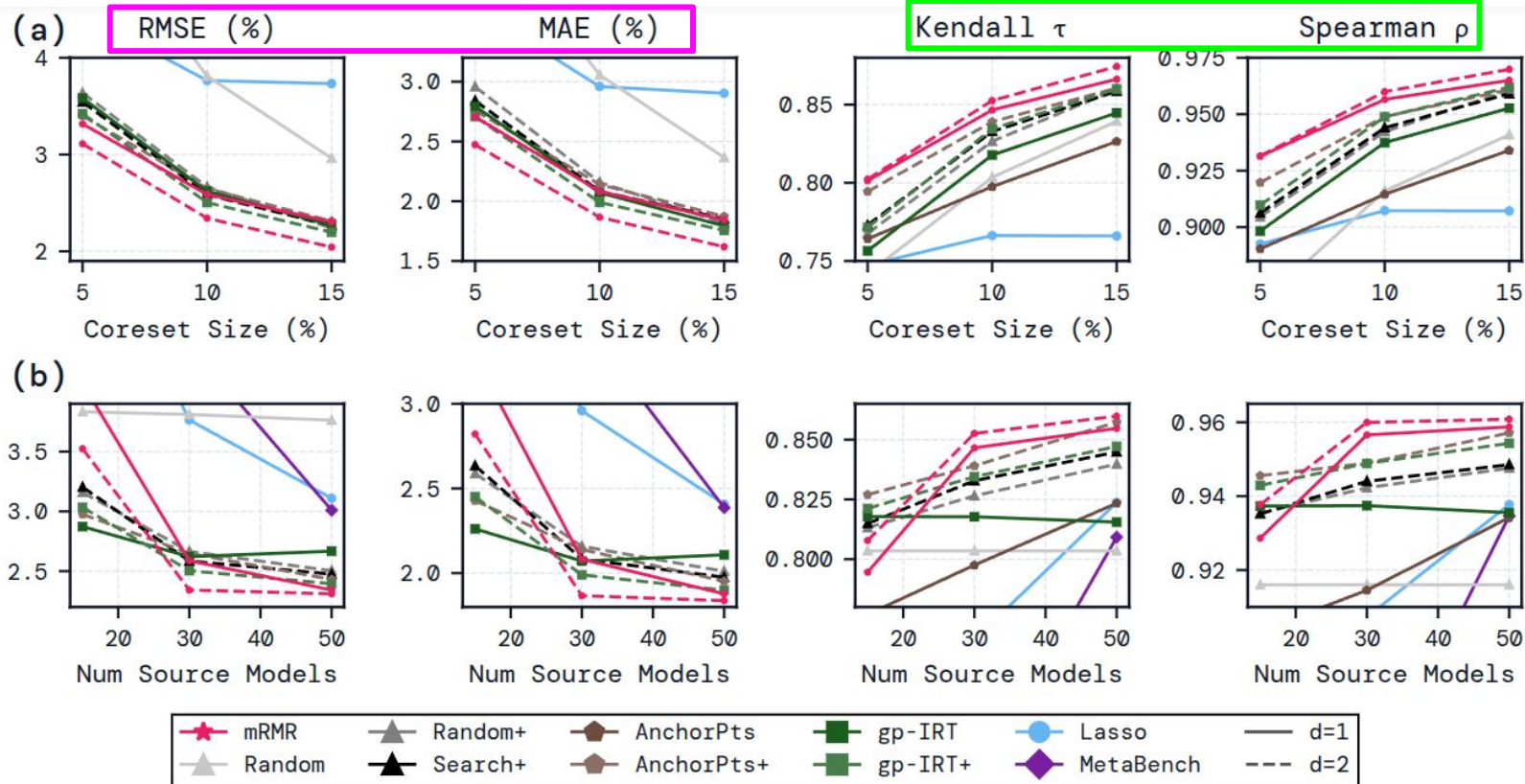


★ mRMR ⬢ AnchorPoints ■ TinyBenchmarks ▲ Baselines + Ridge ++ Kernel Ridge

Binary benchmark results



Binary benchmark results

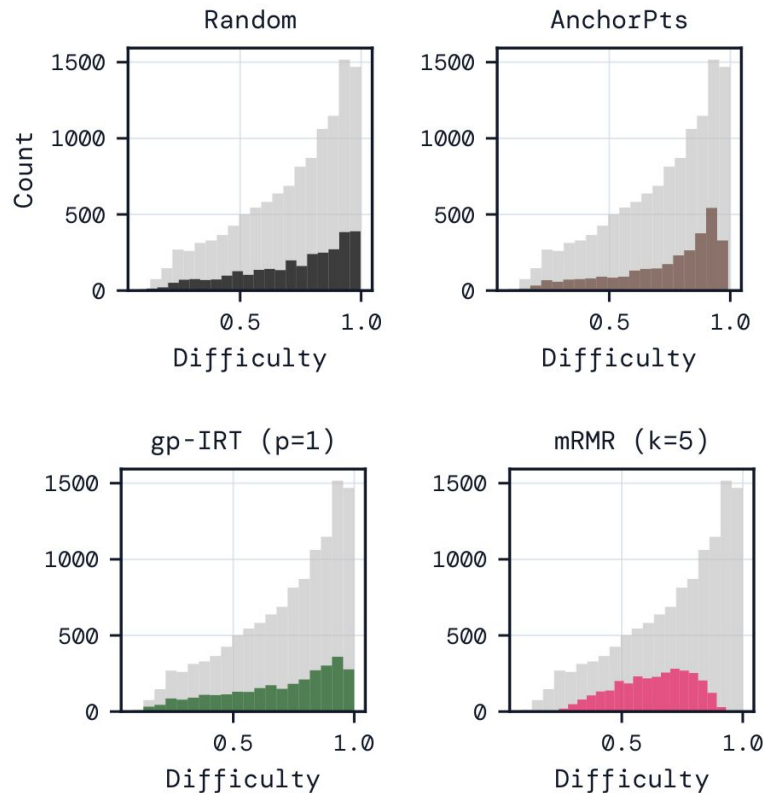


Conclusion

Reframing the ‘efficient benchmarking’ problem with feature selection leads to increased

- Accuracy
- Ranking performance
- Speed
- Coreset stability

Plus, it still only requires per-model per-question scores (and we can get away with pretty small M)



Thanks! Any questions?

Check out the paper on arXiv!



And a Python tutorial on Github!

